

Doubly linked list

1.Insertion at Beginning

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *prev;
7     struct node *next;
8 };
9
10 struct node *head = NULL;
11
12 void beginning(){
13     struct node *first,*second,*third,*newnode;
14
15     first = (struct node*)malloc(sizeof(struct node));
16     second = (struct node*)malloc(sizeof(struct node));
17     third = (struct node*)malloc(sizeof(struct node));
18     newnode = (struct node*)malloc(sizeof(struct node));
19
20     first->data = 10;
21     second->data = 20;
22     third->data = 30;
23     newnode->data = 5;
24
25     head = first;
26
27     first->prev = NULL;
28     first->next = second;
29
30     second->prev = first;
31     second->next = third;
32
33     third->prev = second;
34     third->next = NULL;
35
36     newnode->prev = NULL;
37     newnode->next = head;
38     head->prev = newnode;
39     head = newnode;
40
41 }
42
43 void display(){
44     struct node *temp = head;
45
46     while(temp != NULL){
47         printf("%d ",temp->data);
48         temp = temp->next;
49     }
50 }
51 }
```

list: 5 10 20 30

=== Code Execution Successful ===

main.c

list: 5 10 20 30

=== Code Execution Successful ===

```
36
37     newnode->prev = NULL;
38     newnode->next = head;
39     head->prev = newnode;
40     head = newnode;
41 }
42
43 void display(){
44     struct node *temp = head;
45
46     while(temp != NULL){
47         printf("%d ",temp->data);
48         temp = temp->next;
49     }
50 }
51 }
52
53 int main(){
54     beginning();
55     printf("list: ");
56     display();
57
58     return 0;
59 }
60
61 }
```

list: 5 10 20 30

=== Code Execution Successful ===

2.Insertiom at End

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node{
5      int data;
6      struct node *prev;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void beginning(){
13
14     struct node *first,*second,*third;
15
16     first = (struct node*)malloc(sizeof(struct node));
17     second = (struct node*)malloc(sizeof(struct node));
18     third = (struct node*)malloc(sizeof(struct node));
19
20     first->data = 10;
21     second->data = 20;
22     third->data = 30;
23
24     head = first;
25
26     first->prev = NULL;
```

List: 10 20 30 40

=== Code Execution Successful ===

main.c

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

head = first;

first->prev = NULL;

first->next = second;

second->prev = first;

second->next = third;

third->prev = second;

third->next = NULL;

}

void insertend(){

struct node *temp,*newnode;

newnode = (struct node*)malloc(sizeof(struct node));

newnode->data = 40;

temp = head;

while(temp->next != NULL){

temp = temp->next;

}

Run

Output

List: 10 20 30 40

=== Code Execution Successful ===

main.c

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

temp->next = newnode;

newnode->prev = temp;

newnode->next = NULL;

}

void display(){

struct node *temp = head;

while(temp != NULL){

printf("%d ",temp->data);

temp = temp->next;

}

}

int main(){

beginning();

insertend();

printf("List: ");

display();

return 0;

}

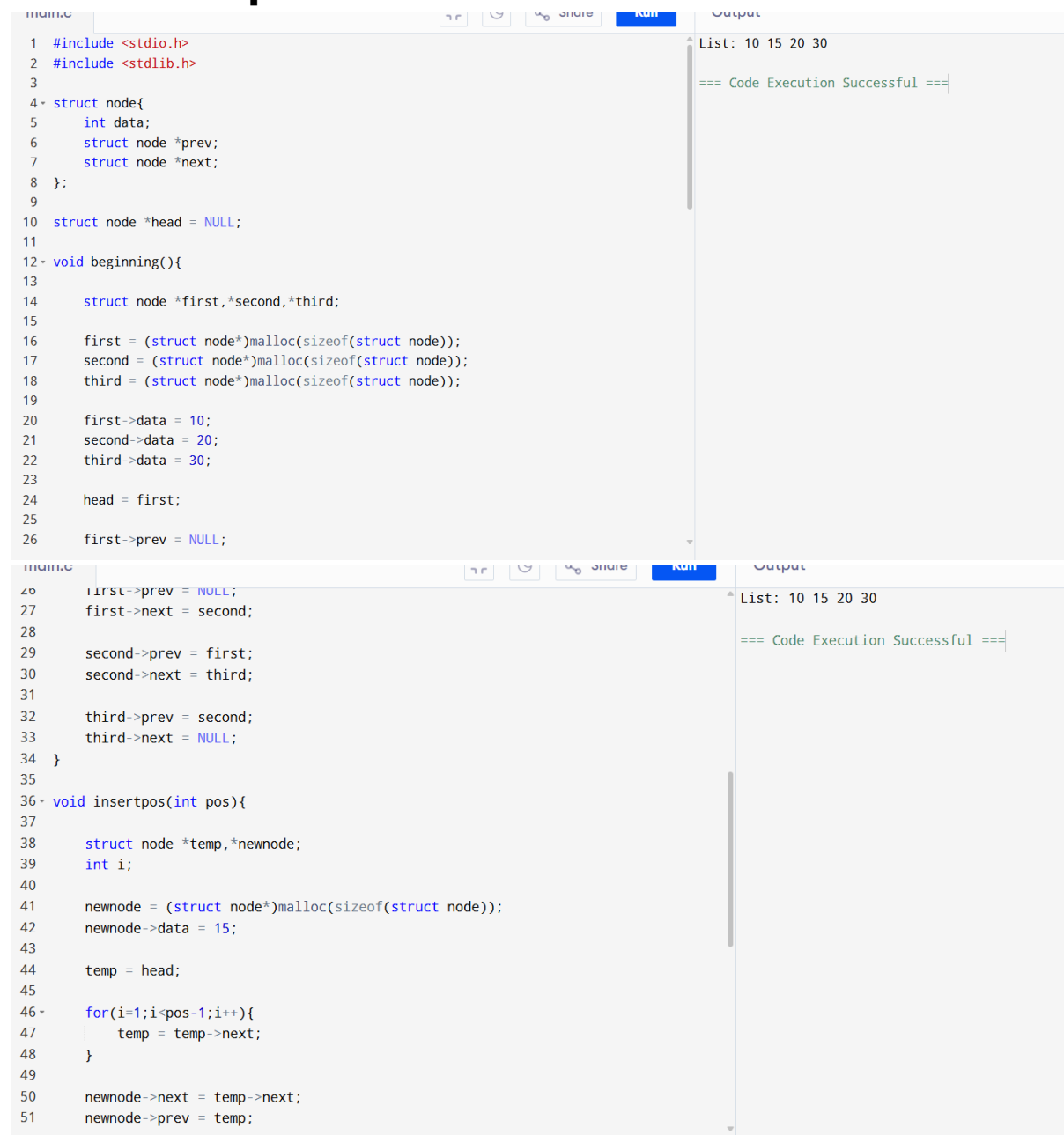
Run

Output

List: 10 20 30 40

=== Code Execution Successful ===

3.Insertion at position



```
main.cpp
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node{
5      int data;
6      struct node *prev;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void beginning(){
13
14     struct node *first,*second,*third;
15
16     first = (struct node*)malloc(sizeof(struct node));
17     second = (struct node*)malloc(sizeof(struct node));
18     third = (struct node*)malloc(sizeof(struct node));
19
20     first->data = 10;
21     second->data = 20;
22     third->data = 30;
23
24     head = first;
25
26     first->prev = NULL;
```

Output

List: 10 15 20 30

=== Code Execution Successful ===

```
main.cpp
26     first->prev = NULL;
27     first->next = second;
28
29     second->prev = first;
30     second->next = third;
31
32     third->prev = second;
33     third->next = NULL;
34 }
35
36 void insertpos(int pos){
37
38     struct node *temp,*newnode;
39     int i;
40
41     newnode = (struct node*)malloc(sizeof(struct node));
42     newnode->data = 15;
43
44     temp = head;
45
46     for(i=1;i<pos-1;i++){
47         temp = temp->next;
48     }
49
50     newnode->next = temp->next;
51     newnode->prev = temp;
```

Output

List: 10 15 20 30

=== Code Execution Successful ===

```
main.c | Run | Output
52
53     temp->next->prev = newnode;
54     temp->next = newnode;
55 }
56
57 void display(){
58     struct node *temp = head;
59
60
61     while(temp != NULL){
62         printf("%d ",temp->data);
63         temp = temp->next;
64     }
65 }
66
67 int main(){
68     beginning();
69
70     insertpos(2);
71
72     printf("List: ");
73     display();
74
75
76     return 0;
77 }
```

List: 10 15 20 30

=== Code Execution Successful ===

4.Deletion at Beginning

```
main.c | Run | Output
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node{
5      int data;
6      struct node *prev;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void beginning(){
13
14     struct node *first,*second,*third;
15
16     first = (struct node*)malloc(sizeof(struct node));
17     second = (struct node*)malloc(sizeof(struct node));
18     third = (struct node*)malloc(sizeof(struct node));
19
20     first->data = 10;
21     second->data = 20;
22     third->data = 30;
23
24     head = first;
25
26     first->prev = NULL;
```

List: 20 30

=== Code Execution Successful ===

```

26     first->prev = NULL;
27     first->next = second;
28
29     second->prev = first;
30     second->next = third;
31
32     third->prev = second;
33     third->next = NULL;
34 }
35
36 void deletebegin(){
37
38     struct node *temp;
39
40     if(head == NULL){
41         printf("List is empty");
42         return;
43     }
44
45     temp = head;
46     head = head->next;
47
48     if(head != NULL){
49         head->prev = NULL;
50     }
51

```

▲ List: 20 30

=== Code Execution Successful ===

```

50 }
51
52 free(temp);
53 }
54
55 void display(){
56
57     struct node *temp = head;
58
59     while(temp != NULL){
60         printf("%d ",temp->data);
61         temp = temp->next;
62     }
63 }
64
65 int main(){
66
67     beginning();
68
69     deletebegin();
70
71     printf("List: ");
72     display();
73
74     return 0;
75 }

```

▲ List: 20 30

=== Code Execution Successful ===

5.Deletion at End

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node{
5      int data;
6      struct node *prev;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void beginning(){
13
14     struct node *first,*second,*third;
15
16     first = (struct node*)malloc(sizeof(struct node));
17     second = (struct node*)malloc(sizeof(struct node));
18     third = (struct node*)malloc(sizeof(struct node));
19
20     first->data = 10;
21     second->data = 20;
22     third->data = 30;
23
24     head = first;
25
26     first->prev = NULL;
```

List: 10 20

=== Code Execution Successful ===

```
26     first->prev = NULL;
27     first->next = second;
28
29     second->prev = first;
30     second->next = third;
31
32     third->prev = second;
33     third->next = NULL;
34 }
35
36 void deleteend(){
37
38     struct node *temp;
39
40     if(head == NULL){
41         printf("List is empty");
42         return;
43     }
44
45     temp = head;
46
47     while(temp->next != NULL){
48         temp = temp->next;
49     }
50
51     temp->prev->next = NULL;
```

List: 10 20

=== Code Execution Successful ===

```
main.c | 51 temp->prev->next = NULL;
52
53 free(temp);
54 }
55
56 void display(){
57     struct node *temp = head;
58
59     while(temp != NULL){
60         printf("%d ",temp->data);
61         temp = temp->next;
62     }
63 }
64 }
65
66 int main(){
67     beginning();
68     deleteend();
69
70     printf("List: ");
71     display();
72
73     return 0;
74 }
75
76 }
```

Output

List: 10 20

=== Code Execution Successful ===

6. Deletion at position

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node{
5     int data;
6     struct node *prev;
7     struct node *next;
8 };
9
10 struct node *head = NULL;
11
12 void beginning(){
13
14     struct node *first,*second,*third;
15
16     first = (struct node*)malloc(sizeof(struct node));
17     second = (struct node*)malloc(sizeof(struct node));
18     third = (struct node*)malloc(sizeof(struct node));
19
20     first->data = 10;
21     second->data = 20;
22     third->data = 30;
23
24     head = first;
25
26     first->prev = NULL;
```

Output

List: 10 30

=== Code Execution Successful ===


```

27     first->next = second;
28
29     second->prev = first;
30     second->next = third;
31
32     third->prev = second;
33     third->next = NULL;
34 }
35
36 void deletepos(int pos){
37
38     struct node *temp;
39     int i;
40
41     if(head == NULL){
42         printf("List is empty");
43         return;
44     }
45
46     temp = head;
47
48     for(i=1;i<pos;i++){
49         temp = temp->next;
50     }
51
52     temp->prev->next = temp->next;

```

▲ List: 10 30

=== Code Execution Successful ===

```

51
52     temp->prev->next = temp->next;
53
54     if(temp->next != NULL){
55         temp->next->prev = temp->prev;
56     }
57
58     free(temp);
59 }
60
61 void display(){
62
63     struct node *temp = head;
64
65     while(temp != NULL){
66         printf("%d ",temp->data);
67         temp = temp->next;
68     }
69 }
70
71 int main(){
72
73     beginning();
74
75     deletepos(2);
76

```

▲ List: 10 30

=== Code Execution Successful ===

```
56     }
57
58     free(temp);
59 }
60
61 void display(){
62     struct node *temp = head;
63
64     while(temp != NULL){
65         printf("%d ",temp->data);
66         temp = temp->next;
67     }
68 }
69
70
71 int main(){
72     beginning();
73
74     deletepos(2);
75
76     printf("List: ");
77     display();
78
79     return 0;
80 }
81 }
```

List: 10 30

=== Code Execution Successful ===